nonessential packages—we don't need them, and we want to keep our image size as small as possible by not installing unnecessary files.

If you're familiar with apt-get in Linux, you may be wondering why we're not running the commands as root with sudo. That's because, by default, commands inside a container are run by the root user, so sudo is unnecessary (although, as we mention on page 192, this has security implications for production apps).

Let's shift gears briefly before we look at the next line of our Dockerfile.

Remember that images, and the containers they spawn, are separate from our local machine—they are isolated, sandboxed environments. Therefore, we need a way to include some of our local files inside the containers we run.

We've already seen in Generating a New Rails App Without Ruby Installed, on page 10, that we can mount a local directory into a running container. A mounted volume acts like a shared directory between the container and the host, and is one way we can make local files accessible inside the container.

However, mounting a volume has a serious downside if it's the only way you get files into a container. Files in a volume aren't part of the image itself; they are overlaid onto the image at *runtime* (when you start a container.) If the mounted files were essential, the image wouldn't function without them, but the whole point of images is to package up everything they need in order to run. Therefore, it's good practice to bake any needed files into the image itself.

The next line in our Dockerfile serves exactly this purpose:

```
COPY . /usr/src/app/
```

This tells Docker to copy all the files from our local, current directory (.) into /usr/src/app on the filesystem of the new image. Since our local, current directory is our Rails root, effectively we're saying, "Copy our Rails app into the container at /usr/src/app." The source path on our local machine is always *relative* to where the Dockerfile is located.

Having added our Rails files into the image at /usr/src/app, we're going to want to run various commands that need to operate in this directory where the files are. For example, soon we'll want to run our app with a Rails server in a container with a command like this:

```
$ docker run [OPTIONS] <our custom image> bin/rails server
```

Unfortunately, this command would fail because, by default, a container's working directory is /, which doesn't contain our Rails app files—we copied those into /usr/src/app.